

CURSO DE C#

CAPÍTULO 3

OPERADORES EN C#

Nivel: Desde principiante hasta nivel práctico

Enfoque: Desarrollo de C#

Objetivo del capítulo

Aprenderás qué son los operadores, cómo se combinan con variables y tipos de datos, cómo se evalúan las expresiones y cómo utilizarlos para calcular, comparar y tomar decisiones. El enfoque avanza desde ejemplos mínimos hasta un caso industrial.

Al finalizar este capítulo serás capaz de:

- Identificar operadores aritméticos, de asignación, comparación, lógicos, incremento/decremento, condicionales, null y bit a bit.
- Construir expresiones correctas y legibles.
- Explicar la diferencia entre `=` y `==`, y entre `&&` y `&`.
- Predecir resultados considerando tipos, conversión y precedencia.
- Aplicar operadores a producción, inventarios, sensores y estados de máquinas.
- Detectar errores de lógica y expresiones ambiguas.

1. Introducción al tema

Un operador es un símbolo o construcción que indica una operación sobre uno o más valores. Por ejemplo, `+` suma y `==` compara. Los valores sobre los que trabaja se llaman operandos y una combinación evaluable de operandos y operadores forma una expresión.

Una analogía: una calculadora recibe datos y una instrucción. En C#, los operadores son esas instrucciones. Sin ellos, las variables podrían almacenar información, pero el programa no podría transformarla ni compararla de forma práctica.

Los operadores aparecen en nóminas, inventarios, videojuegos, sensores, PLC, automatización, robots, drones y aplicaciones empresariales.

2. Operando, operador y expresión

¿Qué es?

Un operando es un dato que participa en una operación. Un operador indica qué hacer con él. Una expresión es una combinación que puede producir un resultado.

```
int piezas = 100;  
int rechazadas = 8;  
int buenas = piezas - rechazadas;
```

piezas y rechazadas son operandos; `-` es el operador; `piezas - rechazadas` es la expresión y `buenas` recibe su resultado: 92.

Sintaxis general

```
operando operador operando
```

3. Operadores aritméticos

Operador	Función	Ejemplo
<code>+</code>	Suma	<code>a + b</code>
<code>-</code>	Resta	<code>a - b</code>
<code>*</code>	Multipliación	<code>a * b</code>
<code>/</code>	División	<code>a / b</code>
<code>%</code>	Residuo	<code>a % b</code>

```
int a = 10;  
int b = 3;  
int cociente = a / b;  
int residuo = a % b;  
double exacto = 10.0 / 3;
```

Con enteros, `10 / 3` produce 3 y `10 % 3` produce 1. Al participar un `double`, la división puede conservar decimales. Esto conecta directamente operadores con tipos de datos.

Ejemplo práctico

```
int fabricadas = 850;
int rechazadas = 25;
int buenas = fabricadas - rechazadas;
double rechazo = (double)rechazadas / fabricadas * 100;
```

Se calculan piezas buenas y porcentaje de rechazo. La conversión a double evita que la división sea una división entera.

4. Operadores de asignación

Operador	Equivale conceptualmente a	Uso
=	asignar	dar un valor inicial
+=	$x = x + n$	acumular
-=	$x = x - n$	reducir
*=	$x = x * n$	multiplicar y guardar
/=	$x = x / n$	dividir y guardar
%=	$x = x \% n$	guardar residuo

```
int stock = 100;
stock += 20;
stock -= 15;
```

Después de += 20 el stock es 120; después de -= 15 es 105.

5. Incremento y decremento

```
int contador = 5;
contador++;
contador--;
++contador;
--contador;
```

++ incrementa una unidad y -- decrementa una unidad. La posición importa cuando se utiliza dentro de otra expresión.

```
int a = 5;
int b = a++; // b=5; después a=6
int c = ++a; // primero a aumenta; c recibe el nuevo valor
```

Para principiantes y código mantenible, separar el incremento de cálculos complejos suele ser más claro.

6. Comparación e igualdad

Operador	Significado
==	igual
!=	diferente
>	mayor
<	menor
>=	mayor o igual
<=	menor o igual

```
double temperatura = 82.5;
bool sobreLimite = temperatura > 80;
```

Una comparación produce bool: true o false. Ese resultado será fundamental en if, while y otras estructuras de control.

7. Operadores lógicos

Operador	Significado	Ejemplo conceptual
&&	AND	activa Y sin alarma

Operador	Significado	Ejemplo conceptual
	OR	alarma 1 O alarma 2
!	NOT	NO activa

```
bool activa = true;
bool sinAlarma = true;
double temperatura = 70;
bool puedeOperar = activa && sinAlarma && temperatura < 80;
```

C# usa cortocircuito para && y ||: si el resultado ya puede determinarse, puede evitar evaluar el resto. Esto es importante al trabajar con condiciones y referencias que podrían no ser válidas.

8. Precedencia y paréntesis

La precedencia establece qué operadores se evalúan primero. Los paréntesis permiten controlar el orden y comunicar la intención.

```
int a = 2 + 3 * 4; // 14
int b = (2 + 3) * 4; // 20
```

La multiplicación se evalúa antes que la suma en el primer caso. No dependas de memorizar toda la tabla de precedencia: en expresiones complejas, usa paréntesis.

9. Operador condicional ?:

```
int piezas = 120;
string estado = piezas >= 100 ? "Meta cumplida" : "Meta pendiente";
```

La condición se evalúa primero. Si es true se selecciona la segunda parte; si es false, la tercera. Es apropiado para decisiones pequeñas y expresiones claras.

10. Operadores relacionados con null

null representa ausencia de una referencia o valor cuando el tipo lo permite. ?? proporciona un valor alternativo; ??= asigna solo cuando el valor actual es null; ?. permite acceso condicional.

```
string? alarma = null;
string mensaje = alarma ?? "Sin alarma";
alarma ??= "Revisar sistema";
```

mensaje obtiene el valor alternativo y después alarma recibe texto porque inicialmente era null.

11. Operadores bit a bit

&, |, ^, ~, << y >> trabajan sobre bits de valores enteros. Se encuentran en protocolos, máscaras, banderas y software de bajo nivel.

```
int valor = 5; // 0101
int mascara = 1; // 0001
int resultado = valor & mascara;
```

El AND bit a bit conserva un bit cuando ambos operandos tienen 1 en esa posición. No debe confundirse con &&, que combina expresiones booleanas.

12. Operadores y tipos de datos

```
int entero = 10;
double decimal = 2.5;
double resultado = entero + decimal;
```

El tipo de los operandos influye en conversiones y resultado. Por eso los operadores no se estudian aislados: forman una unidad con variables y tipos.

13. Ejemplo profesional: monitoreo de máquina

```
int piezas = 1250;
int meta = 1500;
double temperatura = 74.5;
```

```
double limite = 80.0;
bool activa = true;
bool sinAlarma = true;

int faltantes = meta - piezas;
double avance = (double)piezas / meta * 100;
bool puedeProducir = activa && sinAlarma && temperatura < limite;
string estado = puedeProducir ? "PRODUCIENDO" : "DETENIDA";
```

Se combinan resta, división, conversión, comparación, AND lógico y operador condicional. En un sistema real, las variables podrían provenir de sensores, PLC o una base de datos.

14. Tabla de confusiones comunes

Confusión	Diferencia
= / ==	= asigna; == compara.
&& / &	&& es lógico; & es bit a bit.
/	es lógico; puede ser bit a bit.
/ / %	/ obtiene cociente; % obtiene residuo.
++ / +=	++ agrega uno; += agrega una cantidad.
?: / if	?: es una expresión compacta; if permite bloques y flujos más amplios.
?? /	?? trata null; combina booleanos.

Errores de confusión comunes

El error clásico es escribir una asignación cuando se quería comparar. Otro es pensar que toda división produce decimales. También es común usar && cuando se pretende una operación de bits, o hacer expresiones tan largas que el orden de evaluación deja de ser evidente.

Errores comunes al trabajar con operadores

Confundir = con ==. Revisa si quieres guardar un valor o comparar dos valores.

Ignorar división entera. Revisa los tipos antes de dividir.

No usar paréntesis en expresiones complejas. Haz explícita la intención.

Comparar valores de punto flotante suponiendo exactitud matemática absoluta cuando el dominio no lo permite.

Crear condiciones enormes. Divide la lógica en resultados intermedios con nombres claros.

Usar & o | cuando necesitas && o ||.

Mezclar incrementos y otras operaciones en una sola expresión sin necesidad.

Buenas prácticas

- Elige operadores por significado, no solo por brevedad.
- Usa paréntesis cuando mejoren la lectura.
- Verifica tipos y conversiones antes de cálculos.
- Prueba límites: cero, igualdad, negativos y valores máximos.
- Separa cálculos complejos en variables intermedias.

- Usa decimal para muchos importes monetarios cuando la precisión decimal sea requisito.
- Documenta máscaras y operaciones bit a bit.
- Prefiere claridad sobre expresiones ingeniosas difíciles de mantener.

Conectando conocimientos

Los capítulos anteriores aportan variables y tipos de datos. Los operadores permiten ahora transformar y comparar esos valores. Las comparaciones producen bool, que prepara el siguiente paso: estructuras de decisión. Las mismas expresiones aparecerán después en ciclos, métodos, arreglos, clases y proyectos.

Ejercicios prácticos

Ejercicio 1 — Comprensión básica

Declara dos enteros y realiza suma, resta, multiplicación, división y residuo. Muestra cada resultado. Pista: observa qué ocurre con la división.

Ejercicio 2 — Aplicación

Calcula subtotal, descuento y total de una compra. Usa operadores aritméticos y de asignación. No conviertas todo a texto para hacer cálculos.

Ejercicio 3 — Integración

Con variables de una máquina calcula piezas buenas y porcentaje de rechazo, y determina si puede operar combinando condiciones con &&. Pista: separa primero cálculos y decisiones.

Ejercicio 4 — Caso práctico

Crea un inventario con stock actual, entradas, salidas, stock mínimo y estado. Utiliza +=, -=, comparaciones y ?..

Ejercicio 5 — Reto

Evalúa una estación robotizada con producción, temperatura, presión, estado activo y alarma. Calcula indicadores, determina si puede operar y prueba casos límite.

Mini proyecto: Monitor de producción

Contexto

Una línea industrial necesita un programa de consola que calcule indicadores de producción y determine un estado operativo.

Objetivo

Aplicar operadores aritméticos, de asignación, comparación, lógicos y condicionales sobre datos tipados.

Requisitos funcionales

- Registrar meta, piezas fabricadas y rechazadas.
- Calcular piezas buenas y faltantes.
- Calcular porcentajes de avance y rechazo.
- Comparar temperatura con un límite.
- Determinar si la máquina puede operar.
- Mostrar un reporte final.

Requisitos técnicos

- Usar variables de distintos tipos.
- Usar al menos una conversión para un porcentaje.
- Combinar condiciones mediante operadores lógicos.

- Usar paréntesis donde mejoren la claridad.
- Evitar una única expresión gigantesca.

Paso 1

Define las variables de producción y muestra un encabezado.

Paso 2

Calcula piezas buenas y faltantes.

Paso 3

Calcula los porcentajes evitando división entera.

Paso 4

Construye condiciones para temperatura, actividad y alarma.

Paso final

Integra las condiciones, determina el estado y prueba escenarios normales, sobretemperatura, alarma y meta alcanzada.

Comprueba tus conocimientos

1. ¿Qué es un operando?
2. ¿Qué es una expresión?
3. ¿Qué hace %?
4. ¿Cuál es la diferencia entre = y ==?
5. ¿Qué tipo de resultado produce una comparación?
6. ¿Cuál es la diferencia entre && y ||?
7. ¿Qué hace !?
8. ¿Por qué 10 / 3 puede producir 3?
9. ¿Para qué sirve ?:?
10. ¿Para qué sirven ?? y ??=?
11. ¿Qué diferencia existe entre ++ y +=?
12. ¿Por qué usar paréntesis?
13. ¿Qué diferencia existe entre & y &&?
14. ¿Por qué importa el tipo de los operandos?
15. Analiza: int x=5; int y=x++; ¿qué valores quedan?
16. ¿Qué operadores combinarías para verificar que un valor está dentro de un rango?
17. ¿Cómo expresarías que una máquina está activa, sin alarma y por debajo de temperatura límite?

Respuestas y explicaciones

1. Es un valor sobre el que actúa un operador.
2. Es una combinación evaluable que produce un resultado.
3. Obtiene el residuo de una división.
4. = asigna; == compara.
5. bool, true o false.
6. && exige todas las condiciones; || permite que alguna sea verdadera.
7. Invierte un booleano.

8. Porque ambos operandos son enteros y se realiza división entera.
9. Permite elegir entre dos expresiones según una condición.
10. Permiten proporcionar o asignar un valor alternativo cuando existe null.
11. ++ modifica en una unidad; += suma una cantidad elegida y reasigna.
12. Controlan el orden y hacen explícita la intención.
13. & opera sobre bits; && combina booleanos.
14. Porque influye en operaciones, conversiones y resultados.
15. x termina en 6 y y recibe 5.
16. Usaría >= y <= combinados con &&.
17. activa && sinAlarma && temperatura < limite.

Resumen del capítulo

Concepto	Para qué sirve	Error común
Aritméticos	Calcular	Ignorar división entera
Asignación	Modificar valores	Confundir = con ==
Comparación	Generar true/false	Usar operador incorrecto
Lógicos	Combinar condiciones	Confundir && con &
Incremento	Sumar/restar una unidad	Ocultar efectos
Condicional	Elegir un valor	Usarlo para lógica compleja
Null	Manejar ausencia	Confundir null con false
Bit a bit	Manipular bits	Usarlo como lógica booleana

Ideas clave que debes recordar

- Los operadores procesan los datos almacenados en variables.
- El tipo influye en el comportamiento de las operaciones.
- = asigna y == compara.
- Las comparaciones producen bool.
- &&, || y ! permiten construir lógica.
- La división entera puede eliminar la parte decimal.
- Los paréntesis ayudan a controlar y comunicar el orden.
- ?? está relacionado con null, no con condiciones booleanas.
- Los operadores bit a bit tienen un propósito distinto de los lógicos.
- En software profesional, claridad, pruebas y corrección son prioritarias.

Mapa de aprendizaje

Variables y tipos → operadores aritméticos → asignación → comparación → lógica → precedencia → expresiones → decisiones → ciclos → métodos → arreglos → clases → proyectos.

Este capítulo convierte los datos aprendidos anteriormente en operaciones útiles y prepara al estudiante para estructuras de control, donde las comparaciones y expresiones determinarán qué camino sigue el programa.